

*¿Qué es React DOM? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo
descripto, explicando el ejemplo.*

React DOM es la biblioteca encargada de renderizar los componentes de React para el navegador, funciona como puente entre el DOM (Document Object Model) virtual y el DOM del navegador.

La biblioteca react define componentes, estados, hooks, sistema de props, etc, react-dom se encarga de tomar la lógica independiente a la plataforma y mostrarla efectivamente en el navegador.

Características principales:

- Provee el método `createRoot` (versiones modernas) o `ReactDOM.render` (versiones más viejas) para armar la app en un nodo real del HTML.
- Implementa el DOM virtual: una copia liviana en memoria del DOM real que React compara con la versión anterior para actualizar solo lo que cambió, y así evitar re-renderizar toda la página.
- Es específico para entornos web (para apps móviles el equivalente es react native, que usa la misma lógica de React pero pinta sobre componentes nativos en vez de DOM).

```
// index.js
import { createRoot } from 'react-dom/client';
import App from './App';

const contenedor = document.getElementById('root');
const raiz = createRoot(contenedor);
raiz.render(<App />);

// App.js
import { useState } from 'react';

function App() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={() => setContador(contador + 1)}>Subir</button>
      <button onClick={() => setContador(contador - 1)}>Bajar</button>
    </div>
  );
}

export default App;
```

1. `createRoot(contenedor)` toma el `<div id="root">` del HTML real y le dice a React acá va todo el contenido. Ese es el único momento en que React DOM se "conecta" directamente con el DOM del navegador.

2. Cuando hacés clic en un botón, `setContador` actualiza el estado. React arma internamente una nueva versión del DOM virtual con el número actualizado.
3. React DOM compara esa versión nueva con la anterior, se da cuenta de que solo cambió el texto dentro del `<h2>`, y actualiza únicamente ese pedacito en el DOM real no re-crea los botones ni el `<div>` entero.

*¿Qué es React Router? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo
descripto, explicando el ejemplo.*

React Router es una biblioteca externa que permite el manejo de navegación entre distintas vistas en una misma app de una sola página sin recargarla. Es un enrutador multiestrategia que asocia URLs a componentes específicos.

Características principales:

- Permite definir rutas entre una URL y el componente a renderizar.
- Cambia la URL y el contenido visible sin pedirle una nueva página al servidor, lo que hace la app más rápida.
- Permite el anidamiento de rutas, rutas dinámicas, protegidas, redirecciones y navegación programática con `useNavigate`.
- Se compone principalmente de: `BrowserRouter` (provee el contexto de enrutamiento), `Routes` y `Route` (definen qué componente corresponde a cada path), y `Link/NavLink` (para navegar sin recargar la página).

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { BrowserRouter } from 'react-router-dom'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap-icons/font/bootstrap-icons.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
)
```

```

import { Routes, Route } from 'react-router-dom'
import Show from './components/show'
import Create from './components/create'
import Edit from './components/edit'

function App() {
  return (
    <div className="App">
      <Routes>
        <Route path="/" element={<Show />} />
        <Route path="/create" element={<Create />} />
        <Route path="/edit/:id" element={<Edit />} />
      </Routes>
    </div>
  )
}

export default App

```

- 1. BrowserRouter — es el componente que habilita el sistema de enrutamiento en toda la app.
- 2. Routes — es el contenedor que agrupa todas las rutas posibles de esa sección de la app. Su trabajo es mirar la URL actual y decidir cuál de sus Route hijos coincide, para renderizar solo ese.
- 3. Route — asocia una URL con el componente a renderizar. En este ejemplo específico son tres rutas de un CRUD:
 - / → Show
 - /create → Create
 - /edit/:id → Edit